Introduction to brms and Stan

Paul Bürkner

Stan overview

- Probabilistic programming language written in C++ ...
- ... to fit open-ended Bayesian models
- Powerful sampling algorithms: HMC and NUTS
- Automatic differentiation library
- Runs on all major platforms (Windows, OS X, Linux)
- Can be called from R, Python, Julia, Stata, and Matlab

Stan syntax: Linear Regression

```
data {
  int<lower=1> N;  // total number of observations
  vector[N] Y;  // response variable
  int<lower=1> K;  // number of population-level effects
  matrix[N, K] X;  // population-level design matrix
}
parameters {
  vector[K] b;  // population-level effects
  real<lower=0> sigma;  // residual SD
}
model {
  vector[N] mu;
  mu = X * b;
  sigma ~ student_t(3, 0, 10);
  Y ~ normal(mu, sigma);
}
```

Stan syntax: Simple multilevel model by brms (1)

```
// generated with brms 1.6.0
functions {
}
data {
  int<lower=1> N;  // total number of observations
  vector[N] Y;  // response variable
  int<lower=1> K;  // number of population-level effects
  matrix[N, K] X;  // population-level design matrix
  // data for group-level effects of ID 1
  int<lower=1> J_1[N];
  int<lower=1> N_1;
  int<lower=1> M_1;
  vector[N] Z_1_1;
  vector[N] Z_1_2;
  int<lower=1> NC_1;
  int prior_only;  // should the likelihood be ignored?
}
transformed data {
  int Kc;
  matrix[N, K - 1] Xc;  // centered version of X
  vector[K - 1] means_X;  // column means of X before centering
  Kc = K - 1;  // the intercept is removed from the design matrix
  for (i in 2:K) {
    means_X[i - 1] = mean(X[, i]);
    Xc[, i - 1] = X[, i] - means_X[i - 1];
  }
}
```

Stan syntax: Simple multilevel model by brms (2)

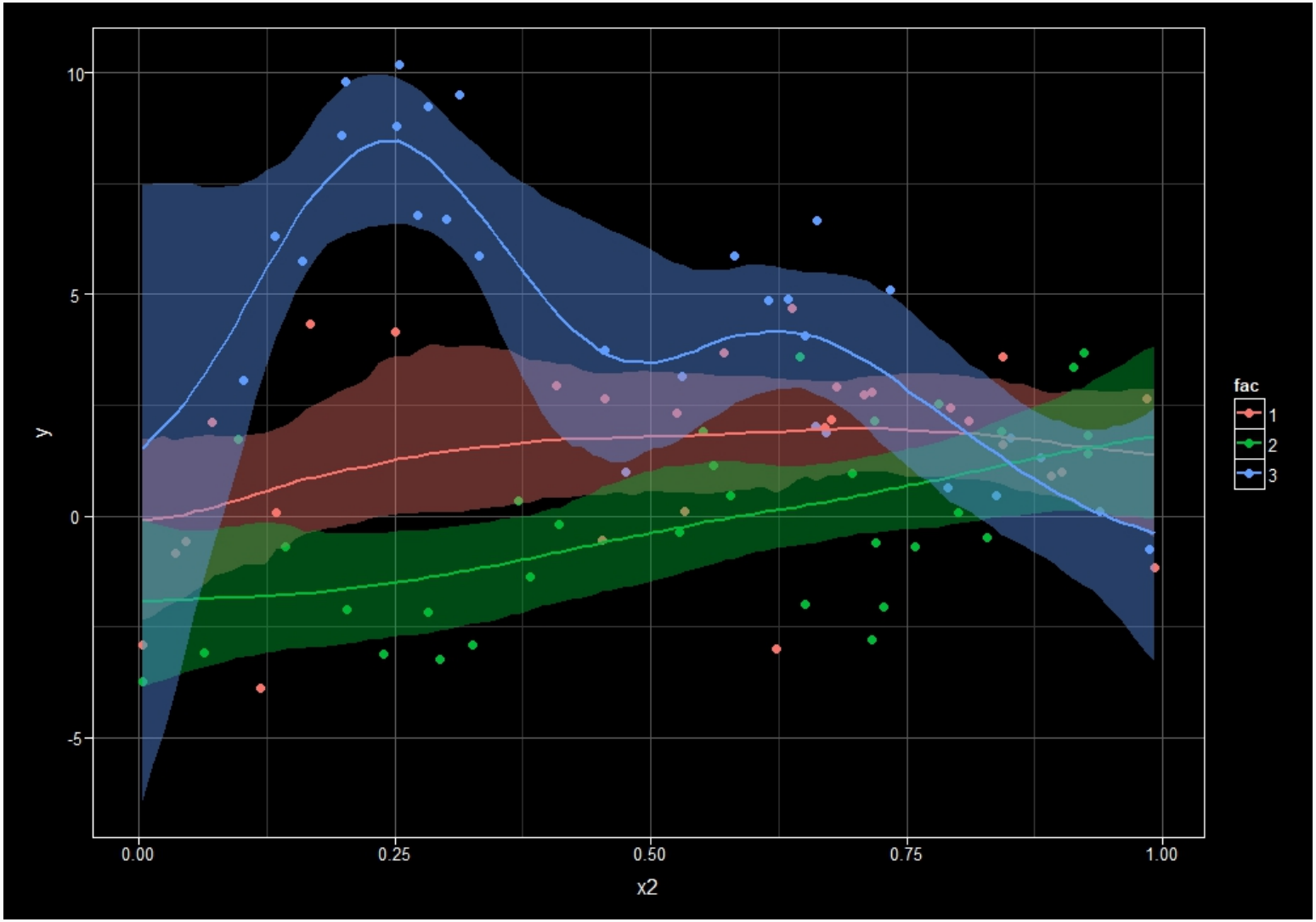
```
parameters {
  vector[Kc] b;  // population-level effects
  real temp_Intercept;  // temporary intercept
  real<lower=0> sigma;  // residual SD
  vector<lower=0>[M_1] sd_1;  // group-level standard deviations
  matrix[M_1, N_1] z_1;  // unscaled group-level effects
  // cholesky factor of correlation matrix
  cholesky_factor_corr[M_1] L_1;
}
transformed parameters {
  // group-level effects
  matrix[N_1, M_1] r_1;
  vector[N_1] r_1_1;
  vector[N_1] r_1_2;
  r_1 = (diag_pre_multiply(sd_1, L_1) * z_1)';
  r_1_1 = r_1[, 1];
  r_1_2 = r_1[, 2];
}
```

Stan syntax: Simple multilevel model by brms (3)

```
model {
  vector[N] mu;
  mu = Xc * b + temp_Intercept;
  for (n in 1:N) {
    mu[n] = mu[n] + (r_1_1[J_1[n]]) * Z_1_1[n] + (r_1_2[J_1[n]]) * Z_1_2[n];
  }
  // prior specifications
  sigma ~ student_t(3, 0, 56);
  sd_1 ~ student_t(3, 0, 56);
  L_1 ~ lkj_corr_cholesky(1);
  to_vector(z_1) ~ normal(0, 1);
  // likelihood contribution
  if (!prior_only) {
    Y ~ normal(mu, sigma);
  }
}
generated quantities {
  real b_Intercept;  // population-level intercept
  corr_matrix[M_1] Cor_1;
  vector<lower=-1,upper=1>[NC_1] cor_1;
  b_Intercept = temp_Intercept - dot_product(means_X, b);
  // take only relevant parts of correlation matrix
  Cor_1 = multiply_lower_tri_self_transpose(L_1);
  cor_1[1] = Cor_1[1,2];
}
```

The brms approach

Fit all kinds of regression models within one unified framework



The brms approach

Make Stan usable for a wider range of audience

- Use R formula syntax
- Internally write Stan code that is readable yet fast
- Provide an easy interface for defining priors
- Facilitate post-processing

brms is offically supported by the Stan Development Team

Some features of brms (1)

Basic model types:

- Linear models
- Count data models
- Categorical and ordinal models
- Survival models
- Response time models
- Quantile Models
- Zero-inflated and hurdle models

Can be extended to:

- Non-linear models
- Mixture models
- Multivariate models

Some features of brms (2)

Other modeling options:

- Group-level terms
- Splines
- Gaussian processes
- Ordinal predictors
- Autocorrelation
- Measurement error
- Censored or truncated responses
- Missing data imputation
- ...

Plans for the future

- Structural equation models
- Improved time series analysis
- Improved survival analysis
- ...

Full list of issues: https://github.com/paul-buerkner/brms/issues

Your ideas are always welcome!

Learn more about brms

- Help within R: `help("brms")`
- Vignettes: `vignette(package = "brms")`
- List of all methods: `methods(class = "brmsfit")`
- Website: https://github.com/paul-buerkner/brms
- Forums: http://discourse.mc-stan.org/
- Contact me: `paul.buerkner@gmail.com`
- Twitter: @paulbuerkner

Publications

- Bürkner P. C. (2017). brms: An R Package for Bayesian Multilevel Models using Stan. *Journal of Statistical Software*. 80(1), 1-28. doi:10.18637/jss.v080.i01
- Bürkner P. C. (in press). Advanced Bayesian Multilevel Modeling with the R Package brms. *The R Journal*.

Learn more about Stan

- Website: http://mc-stan.org/
- Manual: http://mc-stan.org/users/documentation/index.html
- Forums: http://discourse.mc-stan.org/

Selected Publications:

- Bob Carpenter, Andrew Gelman, Matthew D. Hoffman, Daniel Lee, Ben Goodrich, Michael Betancourt, Marcus Brubaker, Jiqiang Guo, Peter Li, and Allen Riddell. 2017. Stan: A probabilistic programming language. *Journal of Statistical Software*. 76(1). 10.18637/jss.v076.i01
- Andrew Gelman, Daniel Lee, and Jiqiang Guo. 2015. Stan: A probabilistic programming language for Bayesian inference and optimization. *Journal of Education and Behavioral Statistics*. 40(5):530–543.